



ÓBUDAI EGYETEM
ÓBUDA UNIVERSITY

Óbudai Egyetem
Neumann János Informatikai Kar

Ármai Zsolt

FILM ADATBÁZIS TERVEZÉSI DOKUMENTÁCIÓ

KONZULENS:
Dr. Simon Nagy Gábor

BUDAPEST, 2025

Tartalomjegyzék

1 Tartalmi kivonat	3
2 Tervezés relációs adatbázisban.....	4
2.1 Egyed-kapcsolat.....	4
2.2 Adatbázis USER létrehozása	4
2.3 Adatbázis táblái és tárolt adatai	4
2.3.1 TITLE_BESTS tábla.....	4
2.3.2 TITLE_FINANCE tábla	5
2.3.3 TITLE_LANGUAGES tábla	5
2.3.4 TITLE_MEDIA tábla	6
2.3.5 TITLE_PRODUCTION tábla elemei	6
2.3.6 TITLE_RATINGS tábla	6
2.3.7 TITLE_TEXT tábla	6
2.3.8 USERS tábla	7
2.3.9 WATCHLIST_ITEMS tábla.....	7
2.3.10 WATCHLISTS tábla	7
2.3.11 Elkészült adatbázis EK diagramja	8
2.4 Szekvenciák és triggerek	8
2.5 Lekérdezések és optimalizáció	9
2.5.1 1. összetett lekérdezés.....	9
2.5.2 2. összetett lekérdezés.....	11
2.5.3 Optimalizáció.....	12
3 Tervezés NoSQL adatbázisban.....	14
3.1 Dokumentumok előállítása relációs adatbázisból	14
3.2 Beágyazott dokumentumok importálása movies dokumentumba	15
3.3 Beágyazott dokumentumok importálása users dokumentumba.....	16
3.4 Lekérdezések	16
3.4.1 Hány film rendelkezik 5-nél nagyobb értékeléssel?	16
3.4.2 Mely műfajok a leggyakoribbak, és hány film tartozik hozzájuk?.....	17
3.4.3 Melyik 3 produkciós iroda készítette a legtöbb filmet?.....	17
3.4.4 Hány felhasználónak van legalább két watchlist-je?	17
3.4.5 Mely 3 film kapta a legtöbb szavazatot?	18
3.4.6 Mely 3 országban gyártották a legtöbb filmet?	18
3.4.7 Mely 3 film hozta a legtöbb profitot?	19
3.4.8 Mely 2 műfaj szerepel a legtöbbet watchlist-eken?.....	19
3.4.9 Mely 3 évben adták ki a legtöbb filmet?.....	20
3.4.10 Mely filmek szerepelnek a legtöbbet watchlist-eken?	20
4 Irodalomjegyzék.....	22
Függelék.....	23
4.1 Schema script-ek.....	23
4.2 Adatok betöltése SQL*Loader-el	25
4.3 TITLE_BASICS tisztítása	27
4.4 USERS generátor	28
4.5 WATCHLIST_ITEMS és WATCHLIST generátor	30

1 Tartalmi kivonat

A feleadat célja egy nagy internetes filmadatbázis tárolási részleteinek kidolgozása relációs és nem relációs adatbáziskezelőkkel.

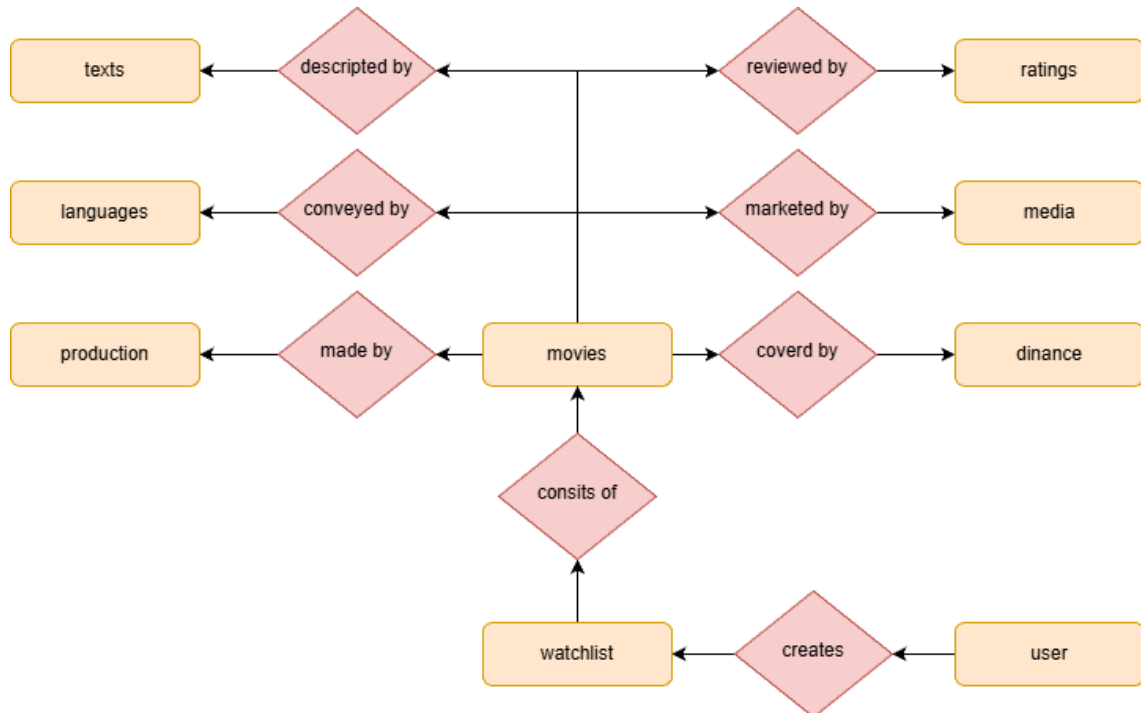
Az adatbázis felhasználói a regisztráció és az ezzel járó adataik megadása után hozzáférnek a filmek adataihoz (általános, pénzügyi, marketing stb.), illetve, listákat (*watchlist*-eket) készíthetnek.

Az relációs adatbázis létrehozása után annak működését két összetett lekérdezésen keresztül szemléltetem, majd ezek után optimalizálom az adatbázist. A relációs adatbázis elkészítése után, azt egy nem relációs (*NoSQL*) adatbázissal is összevetem, majd ezek alapján levonom a következtetéseket.

2 Tervezés relációs adatbázisban

A feladat nem szintetikus adatait a Kaggle [1] felületéről szereztem. Az adatsomag több mint egymillió valós film adatait tartalmazza és a szerző ezeket nem ipari célú felhasználásra ingyenesen rendelkezésre bocsátja.

2.1 Egyed-kapcsolat



1. ábra EK diagram

2.2 Adatbázis USER létrehozása

User elkészítése:

```
CREATE USER MOVIEDB_PROJ IDENTIFIED BY *****
DEFAULT TABLESPACE USERS
TEMPORARY TABLESPACE TEMP
QUOTA UNLIMITED ON USERS
```

Engedélyek megadása:

```
GRANT CONNECT, RESOURCE TO MOVIEDB_PROJ;
```

2.3 Adatbázis táblái és tárolt adatai

Lentebb található a relációs adatbázis táblái és mezői. A táblákat létrehozó scriptek a függelékben láthatók.

2.3.1 TITLE_BESTS tábla

MEZŐ	ADAT TÍPUS	LEÍRÁS
TMDB_ID	NUMBER(10,0)	film egyedi azonosítója [PK]
TITLE	VARCHAR2(300 BYTE)	cím

RELEASE_DATE	DATE	premier időpontja
RUNTIME	NUMBER(5,0)	játékidő
STATUS	VARCHAR2(30 BYTE)	állapot (készül, kiadott)
ORIGINAL_TITLE	VARCHAR2(300 BYTE)	eredeti cím
IMDB_ID	VARCHAR2(15 BYTE)	IMDB azonosító
GENRES	VARCHAR2(500 BYTE)	műfaj

1. táblázat TITLE_BESTS tábla elemei

Az eredeti adathalmaz a következő szűrésen esett át:

- isAdult (Felnőtt) rekordok eltávolítása, majd a teljes oszlop eldobása,
- azon rekordok eldobása melyen VOTE_COUNT-ja (rájuk leadott szavazata) 0.

Az adathalmaz tisztításához Python nyelvet használtam mivel ez a nyelv könnyen használható az ilyen és ehhez hasonló script-ek megírásához. A kód a függelékben megtalálható.

2.3.2 TITLE_FINANCE tábla

Költségvetési adatok filmenként.

MEZŐ	ADAT TÍPUS	LEÍRÁS
TMDB_ID	NUMBER(10,0)	film egyedi azonosítója [PK] [FK]
BUDGET	NUMBER(14,2)	befektetett összeg
REVENUE	NUMBER(14,2)	bevétel
PROFIT	NUMBER	haszon (számított mező)
ROI_PCT	NUMBER	megtérülés (számított mező)

2. táblázat TITLE_FINANCE tábla elemei

Az eredeti adathalmaz a következő módosításon esett át:

- számított mezők hozzáadása. A számítások menete a függelékben megtalálható.

2.3.3 TITLE_LANGUAGES tábla

Filmek nyelvi jellemzői.

MEZŐ	ADAT TÍPUS	LEÍRÁS
TMDB_ID	NUMBER(10,0)	film egyedi azonosítója [PK] [FK]
ORIGINAL_LANGUAGE	VARCHAR2(50 BYTE)	eredeti nyelv
SPOKEN_LANGUAGE	VARCHAR2(100 BYTE)	beszélt nyelvek

3. táblázat TITLE_LANGUAGES tábla elemei

Az adatok nem estek át különösebb változtatáson.

2.3.4 TITLE_MEDIA tábla

Filmek marketing anyagai.

MEZŐ	ADAT TÍPUS	LEÍRÁS
TMDB_ID	NUMBER(10,0)	film egyedi azonosítója [PK] [FK]
HOMEPAGE	VARCHAR2(500 BYTE)	weblap útvonal
POSTER_PATH	VARCHAR2(500 BYTE)	poster kép útvonal
BACKDROP_PATH	VARCHAR2(500 BYTE)	háttér útvonal

4. táblázat TITLE_MEDIA tábla elemei

Az adatok nem estek át különösebb változtatáson.

2.3.5 TITLE_PRODUCTION tábla elemei

Filmek produkciós vállalatai.

MEZŐ	ADAT TÍPUS	LEÍRÁS
TMDB_ID	NUMBER(10,0)	film egyedi azonosítója [PK] [FK]
PRODUCTION_COUNTRY	VARCHAR2(100 BYTE)	vállalat országa
PRODUCTION_COMPANY	VARCHAR2(200 BYTE)	vállalat

5. táblázat TITLE_PRODUCTION tábla elemei

Az adatok nem estek át különösebb változtatáson.

2.3.6 TITLE_RATINGS tábla

A filmek értékelésihez tartozó adatokat tartalmazza.

MEZŐ	ADAT TÍPUS	LEÍRÁS
TMDB_ID	NUMBER(10,0)	film egyedi azonosítója [PK] [FK]
VOTE_COUNT	NUMBER	szavazatok száma
VOTE_AVERAGE	NUMBER(6,3)	értékelések átlaga
POPULARITY	NUMBER(12,4)	népszerűségi mutató

6. táblázat TITLE_RATINGS tábla elemei

Az adatok nem estek át különösebb változtatáson.

2.3.7 TITLE_TEXT tábla

A filmekhez tartozó szöveges adatokat tartalmazza.

MEZŐ	ADAT TÍPUS	LEÍRÁS
TMDB_ID	NUMBER(10,0)	film egyedi azonosítója [PK] [FK]

OVERVIEW	CLOB	összefoglaló
TAGLINE	VARCHAR2(300 BYTE)	szlogen
KEYWORDS	CLOB	kulcsszavak

7. táblázat TITLE_TEXT tábla elemei

Az adatok nem estek át különösebb változtatáson.

2.3.8 USERS tábla

Felhasználók regisztráció során megadott adatai. A tábla szintetikus elemeket tartalmaz, a rekordokat C#-ban írt alkalmazás generálja. A kód megtalálható a függelékek közt.

MEZŐ	ADAT TÍPUS	LEÍRÁS
USER_ID	NUMBER	felhasználó azonosító [PK]
COUNTRY	VARCHAR2(2 BYTE)	ország
SIGNUP_DATE	DATE	regisztráció időpontja
USERNAME	VARCHAR2(50 BYTE)	felhasználónév
EMAIL	VARCHAR2(255 BYTE)	email
NAME	VARCHAR2(100 BYTE)	felhasználó neve
BIRTH_DATE	DATE	születési idő

8. táblázat USERS tábla elemei

2.3.9 WATCHLIST_ITEMS tábla

Felhasználók által összeállított listák filmjei. A tábla szintetikus elemeket tartalmaz, a rekordokat C#-ban írt alkalmazás generálja. A kód megtalálható a függelékek közt.

MEZŐ	ADAT TÍPUS	LEÍRÁS
LIST_ID	NUMBER	lista egyedi azonosítója [PK]
ADDED_AT	TIMESTAMP	listához adás időpontja
POSITION_NO	NUMBER	hely listában [PK]
NOTE	VARCHAR2(500 BYTE)	megjegyzés
TMDB_ID	NUMBER(10,0)	film egyedi azonosítója

9. táblázat WATCHLIST_ITEMS tábla elemei

2.3.10 WATCHLISTS tábla

A felhasználók által generált lejátszási listák. A tábla szintetikus elemeket tartalmaz, a rekordokat C#-ban írt alkalmazás generálja. A kód megtalálható a függelékek közt.

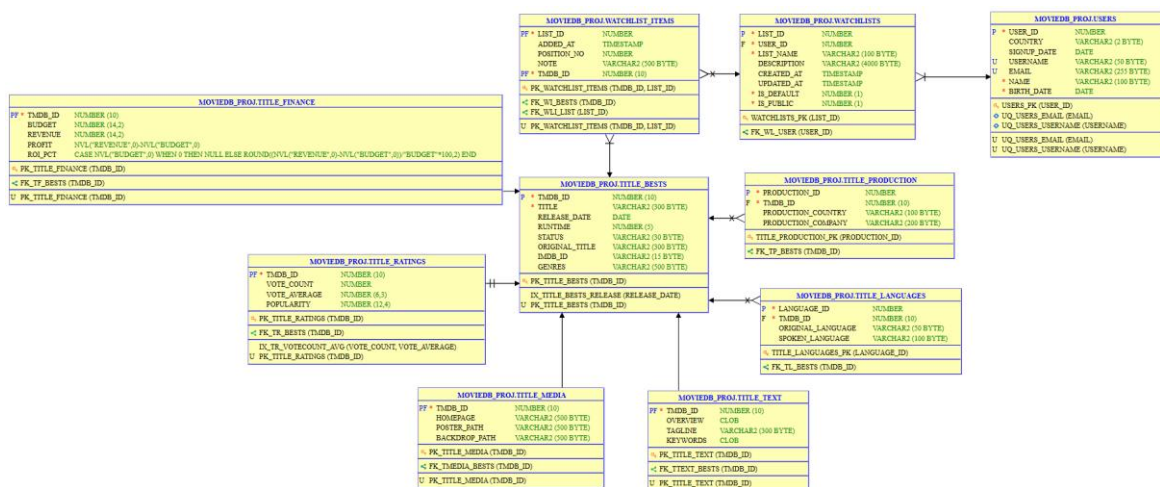
MEZŐ	ADAT TÍPUS	LEÍRÁS
LIST_ID	NUMBER	lista egyedi azonosítója [PK]

USER_ID	NUMBER	felhasználó azonosító [FK]
LIST_NAME	VARCHAR2(100 BYTE)	lista neve
DESCRIPTION	VARCHAR2(4000 BYTE)	lista leírása
CREATED_AT	TIMESTAMP(6)	létrehozás időpontja
UPDATED_AT	TIMESTAMP(6)	utolsó frissítés időpontja
IS_DEFAULT	NUMBER(1,0)	alapértelmezett lista-e
IS_PUBLIC	NUMBER(1,0)	nyilvános lista-e

10. táblázat WATCHLISTS tábla elemei

2.3.11 Elkészült adatbázis EK diagramja

A fentiek alapján a kész adatbázis EK diagramja.



2. ábra Elkészült adatbázis EK diagramja

2.4 Szekvenciák és triggerek

A szintetikus adatokat tartalmazó táblák egyedi azonosítói szekvenciákkal segítségével generáltak. Minden egyes beillesztéskor trigger segítségével a szekvencia következő elem lesz az beillesztendő rekord azonosítója a következő képpen:

```

CREATE SEQUENCE SEQ_USERS START WITH 1 INCREMENT BY 1 NOCACHE;
CREATE OR REPLACE TRIGGER TRG_USERS_PK
BEFORE INSERT ON USERS
FOR EACH ROW
WHEN (NEW.USER_ID IS NULL)
BEGIN
    :NEW.USER_ID := SEQ_USERS.NEXTVAL;
END;
    
```

11. táblázat USERS tábla seq és trigger

```

CREATE SEQUENCE SEQ_WATCHLISTS START WITH 1 INCREMENT BY 1 NOCACHE;
CREATE OR REPLACE TRIGGER TRG_WATCHLISTS_PK
BEFORE INSERT ON WATCHLISTS
FOR EACH ROW
WHEN (NEW.LIST_ID IS NULL)
BEGIN
    :NEW.LIST_ID := SEQ_WATCHLISTS.NEXTVAL;
    
```

```
END;
```

12. táblázat WATCHLISTS tábla seq és trigger

2.5 Lekérdezések és optimalizáció

A következő lekérdezésekkel nyomonkövethető az adatbázis működése, illetve optimalizálható.

2.5.1 1. összetett lekérdezés

A lekérdezés kiírja a közönség számára érdekes (szavazatszám > 20000) és minőségi (átlagos értékelés > 7) filmeket amelyek jó profitot hoztak.

```
SELECT
  b.TMDB_ID,
  b.TITLE,
  EXTRACT(YEAR FROM b.RELEASE_DATE) AS RELEASE_YEAR,
  r.VOTE_AVERAGE,
  r.VOTE_COUNT,
  f.BUDGET,
  f.REVENUE,
  f.PROFIT
FROM
  TITLE_BESTS b
JOIN
  TITLE_RATINGS r ON r.TMDB_ID = b.TMDB_ID
LEFT JOIN
  TITLE_FINANCE f ON f.TMDB_ID = b.TMDB_ID
WHERE
  EXTRACT(YEAR FROM b.RELEASE_DATE) >= 2000
  AND r.VOTE_COUNT >= 20000
  AND r.VOTE_AVERAGE >= 7
ORDER BY
  r.VOTE_AVERAGE DESC,
  (NVL(f.REVENUE,0) - NVL(f.BUDGET,0)) DESC NULLS LAST
```

2.5.1.1 A lekérdezés végrehajtási terve

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
Sort		ORDER BY		16764
Hash Join		OUTER		16764
Access Predicates				16764
F.TMDB_ID(+) = B.TMDB_ID				16764
Nested Loops		OUTER		16764
Statistics Collector				16764
Hash Join				16764
Access Predicates				16764
R.TMDB_ID = B.TMDB_ID				16764
Nested Loops				16764
Statistics Collector				16764
Table Access	TITLE_BESTS	FULL		1214
Filter Predicates				
EXTRACT(YEAR FROM INTERNAL_FUNCTION(B.RELEASE_DATE)) >= 2000				
Table Access	TITLE_RATINGS	BY INDEX ROWID		1
Filter Predicates				
AND				
R.VOTE_AVERAGE >= 7				
R.VOTE_COUNT >= 20000				
Index	PK_TITLE_RATINGS	UNIQUE SCAN		
Access Predicates				
R.TMDB_ID = B.TMDB_ID				
Table Access	TITLE_RATINGS	FULL		41987
Filter Predicates				
AND				
R.VOTE_AVERAGE >= 7				
R.VOTE_COUNT >= 20000				
Table Access	TITLE_FINANCE	BY INDEX ROWID		1
Index	PK_TITLE_FINANCE	UNIQUE SCAN		
Access Predicates				
F.TMDB_ID(+) = B.TMDB_ID				
Table Access	TITLE_FINANCE	FULL		335283

3. ábra 1. összetett lekérdezés végrehajtási terve

A legelső lépés a TITLE_BASICS tábla teljes átvizsgálása. Ez azért történik mert a lekérdezésben függvénnyel nyertük ki a film premier dátumának évét. Ezt követi a TITLE_RATINGS kiszűrése a megadott feltételekkel. Az értékeléseket tartalmazó tábla szűrése ez jóval kevesebb cost-al rendelkezik, mert unique index-elést használ. A két táblát hash-join kapcsolattal köti össze. Végül a TITLE_FINANCE táblával is hash-join kapcsolat létesül és a rendezés is megtörténik.

2.5.1.2 Optimalizáció

A fenti lekérdezés legköltségesebb pontja a TITLE_BESTS tábla full-scan művelete. Annak érdekében, hogy a költségeket csökkentsem a lekérdezés átírásával kezdem.

```
EXTRACT (YEAR FROM b.RELEASE_DATE) >= 2000
```

A fenti sort a következőre cserélem, amitől range-scan eljárásra válthat a keresés:

```
WHERE b.RELEASE_DATE >= DATE '2000-01-01'
```

Illetve akövetkező indexekkel látom a TITLE_BESTS táblát:

```
CREATE INDEX IX_TITLE_BESTS_RELEASE ON TITLE_BESTS (RELEASE_DATE) ;
```

Az új explain plan:

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT				315
SORT				18
HASH JOIN		ORDER BY		315
Access Predicates		OUTER		314
F.TMDB_ID(+) = B.TMDB_ID				
NESTED LOOPS				18
STATISTICS COLLECTOR				314
HASH JOIN				296
Access Predicates				
R.TMDB_ID = B.TMDB_ID				
NESTED LOOPS				18
STATISTICS COLLECTOR				296
TABLE ACCESS	TITLE_RATINGS	FULL		278
Filter Predicates				
AND				
R.VOTE_COUNT >= 20000				
R.VOTE_AVERAGE >= 7				
TABLE ACCESS	TITLE_BESTS	BY INDEX ROWID		1
Filter Predicates				
B.RELEASE_DATE >= TO_DATE('2000-01-01 00:00:00', 'yyyy-mm-dd hh24:miss')				
INDEX	PK_TITLE_BESTS	UNIQUE SCAN		0
Access Predicates				
R.TMDB_ID = B.TMDB_ID				
TABLE ACCESS	TITLE_BESTS	FULL		1
Filter Predicates				
B.RELEASE_DATE >= TO_DATE('2000-01-01 00:00:00', 'yyyy-mm-dd hh24:miss')				
TABLE ACCESS	TITLE_FINANCE	BY INDEX ROWID		1
INDEX	PK_TITLE_FINANCE	UNIQUE SCAN		0
Access Predicates				
F.TMDB_ID(+) = B.TMDB_ID				
TABLE ACCESS	TITLE_FINANCE	FULL		1

4. ábra 1. összetett lekérdezés opcionalizált plan-je

Látható, hogy a legköltségesebb művelet – a TITLE_BESTS teljes átvizsgálása – jóval hatékonyabb az új lekérdezésben, és így a teljes költség is egy nagyságrenddel 1931-ről 315-re csökkenteni.

2.5.2 2. összetett lekérdezés

A 2. összetett lekérdezés visszaadja dinamikus percentilissel meghatározott sikerfilmeket, amelyek legalább 5-ször szerepelnek watchlist-eken.

```

SELECT
  b.TMDB_ID,
  b.TITLE,
  COUNT(*) AS WATCHLIST_MENTIONS
FROM
  WATCHLIST_ITEMS wi
JOIN
  TITLE_BESTS b
  ON b.TMDB_ID = wi.TMDB_ID
WHERE
  EXISTS (
    SELECT 1
    FROM TITLE_RATINGS r
    WHERE r.TMDB_ID = b.TMDB_ID
    AND r.VOTE_COUNT >= 20000
    AND r.VOTE_AVERAGE >= (
      SELECT PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY
r2.VOTE_AVERAGE)
      FROM TITLE_RATINGS r2
      WHERE r2.VOTE_COUNT >= 20000
    )
  )
GROUP BY
  b.TMDB_ID, b.TITLE

```

```

HAVING
COUNT(*) >= 5
ORDER BY
WATCHLIST_MENTIONS DESC, b.TITLE;

```

A fenti lekérdezés explain plan-je:

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT				563
SORT		ORDER BY		563
FILTER				
Filter Predicates				
COUNT(*)>=5				
HASH				
HASH JOIN				
Access Predicates				
GROUP BY				
HASH JOIN				
Access Predicates				
NESTED LOOPS				
STATISTICS COLLECTOR				
HASH JOIN				
Access Predicates				
R.TMDB_ID=B.TMDB_ID				
NESTED LOOPS				
TABLE ACCESS	TITLE_RATINGS	FULL		
Filter Predicates				
AND				
R.VOTE_COUNT>=20000				
R.VOTE_AVERAGE>= (SELECT PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY R2.VOTE_AVERAGE) FROM TITLE_RATINGS R2 WHERE R2.VOTE_COUNT>=20000)				
SORT		GROUP BY		
TABLE ACCESS	TITLE_RATINGS	FULL		
Filter Predicates				
R2.VOTE_COUNT>=20000				
TABLE ACCESS	TITLE_BESTS	BY INDEX ROWID		
INDEX	PK_TITLE_BESTS	UNIQUE SCAN		
Access Predicates				
R.TMDB_ID=B.TMDB_ID				
TABLE ACCESS	TITLE_BESTS	FULL		
INDEX	PK_WATCHLIST_ITEMS	RANGE SCAN		
Access Predicates				
B.TMDB_ID=WT.TMDB_ID				
INDEX	PK_WATCHLIST_ITEMS	FAST FULL SCAN		

5. ábra 2. összetett lekérdezés plan-je

A TITLE_RATINGS-t teljes táblaszkennéssel olvassa és olvasás közben számítja a százalékos és az értékelésre tett feltételeket. Ez a művelet a legköltségesebb a lekérdezésben. A TITLE_BEST és TITLE_RATINGS táblát hash-join-nal kapcsolja, majd a TITLE_BESTS és a WATCHLIST_ITEMS táblát is hash-join-nal kapcsolja és elvégzi a COUNT(*) > 5 feltételt.

2.5.3 Optimalizáció

A fenti lekérdezés optimalizálásaának érdekében új indexet vezetnek be.

```

CREATE INDEX IX_TR_VOTECOUNT_AVG ON TITLE_RATINGS (VOTE_COUNT,
VOTE_AVERAGE) ;

```

A fenti index az 1. összetett lekérdezést is meggyorsítja, így indokoltnak mondható.

Az index bevezetése után az optimált lekérdezés explain plan-je:

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST
SELECT STATEMENT				15
SORT				15
FILTER		ORDER BY		1
Filter Predicates				
COUNT(*) >= 5				
HASH		GROUP BY		1
HASH JOIN				10
Access Predicates				
B.TMDB_ID=W1.TMDB_ID				
NESTED LOOPS				10
STATISTICS COLLECTOR				1
HASH JOIN				3
Access Predicates				
R.TMDB_ID=B.TMDB_ID				
NESTED LOOPS				3
STATISTICS COLLECTOR				1
TABLE ACCESS	TITLE_RATINGS	BY INDEX ROWID BATCHED		3
INDEX	IX_TR_VOTECOUNT_AVG	RANGE SCAN		4
Access Predicates				1
AND				
R.VOTE_COUNT >= 20000				
R.VOTE_AVERAGE >= (SELECT PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY R2.VOTE_AVERAGE) FROM TITLE_RATINGS R2 WHERE R2.VOTE_COUNT IS NOT NULL AND R2.VOTE_COUNT IS NOT NULL				
R.VOTE_COUNT IS NOT NULL				
Filter Predicates				
R.VOTE_AVERAGE >= (SELECT PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY R2.VOTE_AVERAGE) FROM TITLE_RATINGS R2 WHERE R2.VOTE_COUNT IS NOT NULL AND R2.VOTE_COUNT >= 20000				
SORT		GROUP BY		1
INDEX	IX_TR_VOTECOUNT_AVG	RANGE SCAN		60
Access Predicates				3
AND				
R2.VOTE_COUNT >= 20000				
R2.VOTE_COUNT IS NOT NULL				
TABLE ACCESS	TITLE_BESTS	BY INDEX ROWID		1
INDEX	PK_TITLE_BESTS	UNIQUE SCAN		1
Access Predicates				0
R.TMDB_ID=B.TMDB_ID				
TABLE ACCESS	TITLE_BESTS	FULL		1
INDEX	PK_WATCHLIST_ITEMS	RANGE SCAN		1
Access Predicates				1
B.TMDB_ID=W1.TMDB_ID				
INDEX	PK_WATCHLIST_ITEMS	FAST FULL SCAN		1

6. ábra 2. összetett lekérdezés optimált explain plan-je

A költséget így sikerült 563-ról 15-re csökkenteni.

3 Tervezés NoSQL adatbázisban

A NoSQL adatbázis építést MongoDB segítségével végzem, mivel a dokumentumszerű felépítés megfelel az adatok tárolására. Az adatbázis két nagy dokumentuma a USERS és a MOVIES lesz, minden egyéb táblát beágyazott dokumentumként fogunk kezelni a következő képpen:

:: movies	
_id	objectId
finance	{}
budget	int
revenue	int
genres	[]
imdb_id	string
original_title	string
production	{}
production_comp...	(mixed)
production_coun...	(mixed)
ratings	{}
average	(mixed)
popularity	(mixed)
votes	(mixed)
release_date	date
runtime	int
status	string
title	string
tmdb_id	int

:: users	
_id	objectId
birth_date	date
country	string
email	string
name	string
signup_date	date
user_id	int
username	string
watchlists	[]
created_at	string
description	string
is_default	bool
is_public	bool
items	[]
added_at	string
note	string
position	null
tmdb_id	int
list_id	int
name	string
updated_at	string

7. ábra Dokumentumok felépítése MongoDB-ben

3.1 Dokumentumok előállítás relációs adatbázisból

Annak érdekében, hogy a relációs adatbázis rekordjait átvihessem MongoDB-be elsőként exportáltam a táblákat SQL Developer segítségével JSON formátumba.

A JSON dokumentumok kezeléséhez és az adatbázis létrehozásához telepítettem a mongodb, mongosh (mongodb shell) és mongodb tools eszközöket. Ezek után a parancssoros felületről a következő parancs futtatásával az elsődleges dokumentumok importálhatók:

```
mongoimport --db moviedb_proj --collection stg_title_bests --type json --file "F:\title_bests.json" --jsonArray
```

Az összes egyéb tábla adatainak importálása hasonló képpen történt a JSON elérésének és nevének változtatásával.

Az importálásokat követően azon mezőket amelyek dátumok tartalmazznak a következő módon alakítottam megfelelő adatformátumúvá:

```
db.movies.updateMany (
```

```

{
  release_date: { $type: "string" },
  release_date: { $regex: /^\\d{4}-\\d{2}-\\d{2}$/ }
},
[
  { $set: { release_date: { $toDate: "$release_date" } } }
]
)

```

3.2 Beágyazott dokumentumok importálása movies dokumentumba

A rekordok nagy száma miatt bulkWrite módszerrel kellett beágyaznom a dokumentumokat, a következő képpen:

```

const BATCH = 2000;
let ops = [];
let n = 0;

const cur = db.stg_title_ratings.find(
  {},
  { _id: 0, tmdb_id: 1, vote_average: 1, vote_count: 1, popularity: 1 }
);

while (cur.hasNext()) {
  const d = cur.next();

  ops.push({
    updateOne: {
      filter: { tmdb_id: d.tmdb_id },
      update: {
        $set: {
          "ratings.average": Number(d.vote_average),
          "ratings.votes": Number(d.vote_count),
          "ratings.popularity": Number(d.popularity)
        }
      }
    }
  });

  if (ops.length === BATCH) {
    db.movies.bulkWrite(ops, { ordered: false });
    ops = [];
    n++;
  }
  if (ops.length) db.movies.bulkWrite(ops, { ordered: false });
  print("Frissítve ennyi rekord:", n);
}

```

Egyéb beágyazott dokumentumok hasonlóan kerültek hozzáadásra, mezőnevek változtatásával.

3.3 Beágyazott dokumentumok importálása users dokumentumba

A rekordok nagy száma miatt itt is bulkWrite metódust kellett használnom:

```
const BATCH = 2000;
let ops = [];
let n = 0;

const cur = db.stg_watchlists.find(
  {},
  { _id: 0, user_id: 1, list_id: 1, list_name: 1, description: 1,
    is_default: 1, is_public: 1, created_at: 1, updated_at: 1 }
);

while (cur.hasNext()) {
  const d = cur.next();

  ops.push({
    updateOne: {
      filter: { user_id: d.user_id },
      update: {
        $push: {
          watchlists: {
            list_id: d.list_id,
            name: d.list_name,
            description: d.description ?? "",
            is_default: !!d.is_default,
            is_public: !!d.is_public,
            created_at: d.created_at || null,
            updated_at: d.updated_at || null,
            items: []
          }
        }
      }
    }
  });

  if (ops.length === BATCH) { db.users.bulkWrite(ops, { ordered:
false }); ops = []; }
  n++;
}
if (ops.length) db.users.bulkWrite(ops, { ordered: false });

print("Hozzáadva ennyi rekord:", n);
```

Egyéb beágyazott dokumentumok hasonlóan kerültek hozzáadásra a mezőnevek változtatásával.

3.4 Lekérdezések

3.4.1 Hány film rendelkezik 5-nél nagyobb értékeléssel?

```
db.movies.countDocuments({ "ratings.average": { $lt: 5 } })
> 7110
```

3.4.2 Mely műfajok a leggyakoribbak, és hány film tartozik hozzájuk?

```
db.movies.aggregate([
  { $unwind: "$genres" },
  { $group: { _id: "$genres", film_count: { $sum: 1 } } },
  { $sort: { film_count: -1 } },
  { $limit: 5 }
])
```

Lekérdezés eredménye:

```
{
  _id: 'Drama',
  film_count: 109450
}
{
  _id: 'Comedy',
  film_count: 75613
}
{
  _id: 'Documentary',
  film_count: 49729
}
{
  _id: 'Romance',
  film_count: 30392
}
{
  _id: 'Thriller',
  film_count: 29142
}
```

3.4.3 Melyik 3 produkciós iroda készítette a legtöbb filmet?

Lekérdezés:

```
db.movies.aggregate([
  { $group: { _id: "$production.production_company", film_count: {
    $sum: 1 } } },
  { $sort: { film_count: -1 } },
  { $limit: 3 }
])
```

Eredmény:

```
{
  _id: 'Metro-Goldwyn-Mayer',
  film_count: 1606
}
{
  _id: 'Warner Bros. Pictures',
  film_count: 1084
}
{
  _id: 'Paramount',
  film_count: 1029
}
```

3.4.4 Hány felhasználónak van legalább két watchlist-je?

Lekérdezés:

```
db.users.countDocuments({ "watchlists.1": { $exists: true } })
```

Eredmény:

>669

3.4.5 Mely 3 film kapta a legtöbb szavazatot?

Lekérdezés:

```
db.movies.find(
  { "ratings.votes": { $exists: true } },
  { _id: 0, title: 1, "ratings.votes": 1, "ratings.average": 1 }
).sort({ "ratings.votes": -1 }).limit(3)
```

Eredmény:

```
{
  title: 'Inception',
  ratings: {
    average: 8.364,
    votes: 34495
  }
}
{
  title: 'Interstellar',
  ratings: {
    average: 8.417,
    votes: 32571
  }
}
{
  title: 'The Dark Knight',
  ratings: {
    average: 8.512,
    votes: 30619
  }
}
```

3.4.6 Mely 3 országban gyártották a legtöbb filmet?

Lekérdezés:

```
db.movies.aggregate([
  { $group: { _id: "$production.production_country", film_count: {
    $sum: 1 } } },
  { $sort: { film_count: -1 } },
  { $limit: 4 }
])
```

Eredmény:

```
{
  _id: 'United States of America',
  film_count: 72132
}
{
  _id: 'France',
  film_count: 15270
}
{
  _id: 'United Kingdom',
  film_count: 13848
}
```

```
}
```

3.4.7 Mely 3 film hozta a legtöbb profitot?

Lekérdezés:

```
db.movies.aggregate([
  { $project: { title: 1, profit: { $subtract: ["$finance.revenue",
"$finance.budget"] } } },
  { $sort: { profit: -1 } },
  { $limit: 3 }
])
```

Eredmény:

```
{
  _id: ObjectId('68fe5c7ba20fab8422c5517a'),
  title: 'Avatar',
  profit: 2686706026
}
{
  _id: ObjectId('68fe5c7ba20fab8422c55185'),
  title: 'Avengers: Endgame',
  profit: 2444000000
}
{
  _id: ObjectId('68fe5c7ba20fab8422c5518a'),
  title: 'Titanic',
  profit: 2064162353
}
```

3.4.8 Mely 2 műfaj szerepel a legtöbbet watchlist-eken?

Lekérdezés:

```
db.users.aggregate([
  { $unwind: "$watchlists" },
  { $unwind: "$watchlists.items" },
  {
    $lookup: {
      from: "movies",
      localField: "watchlists.items.tmdb_id",
      foreignField: "tmdb_id",
      as: "movie"
    }
  },
  { $unwind: "$movie" },
  { $unwind: "$movie.genres" },
  { $group: { _id: "$movie.genres", count: { $sum: 1 } } },
  { $sort: { count: -1 } },
  { $limit: 2 }
])
```

Eredmény:

```
{
  _id: 'Drama',
  count: 22700
}
```

```
{
  _id: 'Comedy',
  count: 18556
}
```

3.4.9 Mely 3 évben adták ki a legtöbb filmet?

Lekérdezés:

```
db.movies.aggregate([
  { $addFields: { year: { $year: "$release_date" } } },
  { $group: { _id: "$year", film_count: { $sum: 1 } } },
  { $sort: { film_count: -1 } },
  { $limit: 3 }
])
```

Eredmény:

```
{
  _id: 2019,
  film_count: 12628
}
{
  _id: 2018,
  film_count: 12132
}
{
  _id: 2017,
  film_count: 12109
}
```

3.4.10 Mely filmek szerepelnek a legtöbbet watchlist-eken?

Lekérdezés:

```
db.users.aggregate([
  { $unwind: "$watchlists" },
  { $unwind: "$watchlists.items" },
  { $group: { _id: "$watchlists.items.tmdb_id", count: { $sum: 1 } } },
],
{ $sort: { count: -1 } },
{ $limit: 5 },
{
  $lookup: {
    from: "movies",
    localField: "_id",
    foreignField: "tmdb_id",
    as: "movie"
  }
},
{ $unwind: "$movie" },
{ $project: { _id: 0, title: "$movie.title", count: 1 } }
])
```

Eredmény:

```
{
  count: 14,
  title: 'The Diving Bell and the Butterfly'
}
```

```
{
  count: 14,
  title: 'Gamer'
}
{
  count: 14,
  title: 'Tammy'
}
{
  count: 14,
  title: 'Red Lights'
}
{
  count: 14,
  title: 'Deep Water'
}
```

A fenti eredmények azonos gyakorisága a WATCHLIST_ITEMS generátor script sajátosságából fakad.

4 Irodalomjegyzék

- [1] <https://www.kaggle.com/>, „Kaggle,” 25 október 2025. [Online]. Available: <https://www.kaggle.com/datasets/asaniczka/tmdb-movies-dataset-2023-930k-movies>. [Hozzáférés dátuma: 25 október 2025].

Függelék

4.1 Schema script-ek

```
CREATE TABLE TITLE_BESTS (
  TMDB_ID          NUMBER(10)          PRIMARY KEY,
  TITLE            VARCHAR2(300 BYTE) NOT NULL,
  RELEASE_DATE     DATE,
  RUNTIME          NUMBER(4),
  STATUS           VARCHAR2(30),
  ORIGINAL_TITLE   VARCHAR2(300 BYTE),
  IMDB_ID          VARCHAR2(15 BYTE),
);

CREATE TABLE TITLE_RATINGS (
  TMDB_ID          NUMBER(10)          PRIMARY KEY,
  VOTE_COUNT       NUMBER,
  VOTE_AVERAGE    NUMBER(6,3),
  POPULARITY       NUMBER(12,4)
);

CREATE TABLE TITLE_FINANCE (
  TMDB_ID          NUMBER(10)          PRIMARY KEY,
  BUDGET           NUMBER(14,2),
  REVENUE          NUMBER(14,2),
  PROFIT           GENERATED ALWAYS AS (NVL(REVENUE,0) - NVL(BUDGET,0))
VIRTUAL,
  ROI_PCT          GENERATED ALWAYS AS (
    CASE
      WHEN NVL(BUDGET,0) = 0 THEN NULL
      ELSE ROUND( (NVL(REVENUE,0) - NVL(BUDGET,0)) /
        BUDGET * 100, 2 )
    END
  ) VIRTUAL,

  CONSTRAINT FK_TF_BESTS
    FOREIGN KEY (TMDB_ID)
    REFERENCES TITLE_BESTS (TMDB_ID)
    ON DELETE CASCADE);

CREATE TABLE TITLE_PRODUCTION (
  TMDB_ID          NUMBER(10)          PRIMARY KEY,
  PRODUCTION_COUNTRY VARCHAR2(100 BYTE),
  PRODUCTION_COMPANY VARCHAR2(200 BYTE),

  CONSTRAINT FK_TP_BESTS
    FOREIGN KEY (TMDB_ID)
    REFERENCES TITLE_BESTS (TMDB_ID)
    ON DELETE CASCADE);

CREATE TABLE TITLE_LANGUAGES (
  TMDB_ID          NUMBER(10)          PRIMARY KEY,
  ORIGINAL_LANGUAGE VARCHAR2(50 BYTE),
  SPOKEN_LANGUAGE  VARCHAR2(100 BYTE),
```

```

CONSTRAINT FK_TL_BESTS
  FOREIGN KEY (TMDB_ID)
  REFERENCES TITLE_BESTS (TMDB_ID)
  ON DELETE CASCADE);

CREATE TABLE TITLE_TEXT (
  TMDB_ID    NUMBER(10)    PRIMARY KEY,
  OVERVIEW   CLOB,
  TAGLINE    VARCHAR2(300 BYTE),
  KEYWORDS   CLOB,

  CONSTRAINT FK_TTEXT_BESTS
    FOREIGN KEY (TMDB_ID)
    REFERENCES TITLE_BESTS (TMDB_ID)
    ON DELETE CASCADE
);

CREATE TABLE TITLE_MEDIA (
  TMDB_ID    NUMBER(10)    PRIMARY KEY,
  BACKDROP_PATH VARCHAR2(500 BYTE),
  POSTER_PATH  VARCHAR2(500 BYTE),
  HOMEPAGE     VARCHAR2(500 BYTE),

  CONSTRAINT FK_TMEDIA_BESTS
    FOREIGN KEY (TMDB_ID)
    REFERENCES TITLE_BESTS (TMDB_ID)
    ON DELETE CASCADE);

CREATE TABLE USERS (
  USER_ID NUMBER PRIMARY KEY,
  COUNTRY VARCHAR(2) NOT NULL,
  SIGNUP_DATE DATE NOT NULL,
  BIRTH_DATE DATE NOT NULL,
  EMAIL VARCHAR2(255) NOT NULL,
  NAME VARCHAR2(100) NOT NULL,
);

ALTER TABLE USERS ADD CONSTRAINT CHK_USERS_COUNTRY_LENGTH CHECK
(LENGTH(COUNTRY) = 2);

ALTER TABLE USERS ADD CONSTRAINT CHK_USERS_BIRTH_YEAR CHECK
(BIRTH_YEAR BETWEEN 1900 AND EXTRACT(YEAR FROM SYSDATE));

ALTER TABLE USERS ADD CONSTRAINT CHK_USERS_SIGNUP_DATE CHECK
(SIGNUP_DATE <= SYSDATE);

CREATE TABLE WATCHLISTS (
  LIST_ID NUMBER PRIMARY KEY,
  USER_ID NUMBER NOT NULL,
  LIST_NAME VARCHAR2(100) NOT NULL,
  DESCRIPTION VARCHAR2(4000),
  IS_DEFAULT CHAR(1) DEFAULT 'N' CHECK (IS_DEFAULT IN ('Y', 'N')),
  IS_PUBLIC CHAR(1) DEFAULT 'N' CHECK (IS_PUBLIC IN ('Y', 'N')),
  CREATED_AT TIMESTAMP DEFAULT SYSTIMESTAMP,
  UPDATED_AT TIMESTAMP,
  CONSTRAINT FK_WL_USER FOREIGN KEY (USER_ID) REFERENCES
USERS (USER_ID)
);

```

```

CREATE TABLE WATCHLIST_ITEMS (
  LIST_ID NUMBER NOT NULL,
  TMDB_ID VARCHAR2(12) NOT NULL,
  ADDED_AT TIMESTAMP DEFAULT SYSTIMESTAMP,
  POSITION_NO NUMBER,
  NOTE VARCHAR2(500),
  CONSTRAINT PK_WLI PRIMARY KEY (LIST_ID, TCONST),
  CONSTRAINT FK_WLI_LIST FOREIGN KEY (LIST_ID) REFERENCES
WATCHLISTS (LIST_ID) ON DELETE CASCADE,
  CONSTRAINT FK_WLI_TITLE FOREIGN KEY (TMDB_ID) REFERENCES
TITLE_BASICS (TMDB_ID),
  CONSTRAINT CK_WLI_POS CHECK (POSITION_NO IS NULL OR POSITION_NO >=
1)
);

```

4.2 Adatok betöltése SQL*Loader-el

```

OPTIONS (SKIP=1, ERRORS=999999, DIRECT=TRUE)
LOAD DATA
CHARACTERSET UTF8
INFILE 'title_bests.csv'
BADFILE 'title_bests.bad'
DISCARDFILE 'title_bests.dis'
INTO TABLE TITLE_BESTS
APPEND
FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'
TRAILING NULLCOLS
(
  TMDB_ID          INTEGER EXTERNAL      ,
  TITLE            CHAR(300)             NULLIF TITLE=BLANKS,
  RELEASE_DATE     CHAR(20)              NULLIF RELEASE_DATE=BLANKS
                                "CASE
                                WHEN REGEXP_LIKE (:RELEASE_DATE, '^([0-9]{4}-
[0-9]{2}-[0-9]{2})$')
                                THEN TO_DATE (:RELEASE_DATE, 'YYYY-MM-DD')
                                ELSE NULL
                                END",
  RUNTIME          INTEGER EXTERNAL      NULLIF RUNTIME=BLANKS,
  STATUS           CHAR(30)              NULLIF STATUS=BLANKS,
  ORIGINAL_TITLE   CHAR(300)             NULLIF ORIGINAL_TITLE=BLANKS,
  IMDB_ID          CHAR(15)              NULLIF IMDB_ID=BLANKS,
  GENRES           CHAR(500)             NULLIF GENRES=BLANKS
)

```

```

OPTIONS (SKIP=1, ERRORS=999999, DIRECT=TRUE)
LOAD DATA
CHARACTERSET UTF8
INFILE 'title_ratings.csv'
BADFILE 'title_ratings.bad'
DISCARDFILE 'title_ratings.dis'
INTO TABLE TITLE_RATINGS
APPEND
FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'
TRAILING NULLCOLS
(
  TMDB_ID          INTEGER EXTERNAL,

```

```

VOTE_COUNT      INTEGER EXTERNAL NULLIF VOTE_COUNT=BLANKS,
VOTE_AVERAGE    "TO_NUMBER(:VOTE_AVERAGE,      '99D999',
'NLS_NUMERIC_CHARACTERS = '','')",

```

```

OPTIONS (SKIP=1, ERRORS=999999, DIRECT=TRUE)

```

```

LOAD DATA

```

```

CHARACTERSET UTF8

```

```

INFILE 'title_finance.csv'

```

```

BADFILE 'title_finance.bad'

```

```

DISCARDFILE 'title_finance.dis'

```

```

INTO TABLE TITLE_FINANCE

```

```

APPEND

```

```

FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'

```

```

TRAILING NULLCOLS

```

```

(
  TMDB_ID      INTEGER EXTERNAL,
  BUDGET        INTEGER EXTERNAL NULLIF BUDGET=BLANKS,
  REVENUE        INTEGER EXTERNAL NULLIF REVENUE=BLANKS
)

```

```

OPTIONS (SKIP=1, ERRORS=999999, DIRECT=TRUE)

```

```

LOAD DATA

```

```

CHARACTERSET UTF8

```

```

INFILE 'title_production.csv'

```

```

BADFILE 'title_production.bad'

```

```

DISCARDFILE 'title_production.dis'

```

```

INTO TABLE TITLE_PRODUCTION

```

```

APPEND

```

```

FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'

```

```

TRAILING NULLCOLS

```

```

(
  TMDB_ID      INTEGER EXTERNAL,
  PRODUCTION_COUNTRY CHAR(100) NULLIF PRODUCTION_COUNTRY=BLANKS,
  PRODUCTION_COMPANY CHAR(200) NULLIF PRODUCTION_COMPANY=BLANKS
)

```

```

OPTIONS (SKIP=1, ERRORS=999999, DIRECT=TRUE)

```

```

LOAD DATA

```

```

CHARACTERSET UTF8

```

```

INFILE 'title_languages.csv'

```

```

BADFILE 'title_languages.bad'

```

```

DISCARDFILE 'title_languages.dis'

```

```

INTO TABLE TITLE_LANGUAGES

```

```

APPEND

```

```

FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"'

```

```

TRAILING NULLCOLS

```

```

(
  TMDB_ID      INTEGER EXTERNAL,
  ORIGINAL_LANGUAGE CHAR(50) NULLIF ORIGINAL_LANGUAGE=BLANKS,
  SPOKEN_LANGUAGE CHAR(100) NULLIF SPOKEN_LANGUAGE=BLANKS
)

```

```

OPTIONS (SKIP=1, ERRORS=999999, DIRECT=FALSE)

```

```

LOAD DATA

```

```

CHARACTERSET UTF8

```

```

INFILE 'title_text.csv'

```

```

BADFILE 'title_text.bad'

```

```

DISCARDFILE 'title_text.dis'
INTO TABLE TITLE_TEXT
APPEND
FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY ''
TRAILING NULLCOLS
(
    TMDB_ID      INTEGER EXTERNAL,
    OVERVIEW     CHAR(32767)    NULLIF OVERVIEW=BLANKS,
    TAGLINE      CHAR(300)      NULLIF TAGLINE=BLANKS,
    KEYWORDS     CHAR(32767)    NULLIF KEYWORDS=BLANKS
)

OPTIONS (SKIP=1, ERRORS=999999, DIRECT=TRUE)
LOAD DATA
CHARACTERSET UTF8
INFILE 'title_media.csv'
BADFILE 'title_media.bad'
DISCARDFILE 'title_media.dis'
INTO TABLE TITLE_MEDIA
APPEND
FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY ''
TRAILING NULLCOLS
(
    TMDB_ID      INTEGER EXTERNAL,
    BACKDROP_PATH CHAR(500)    NULLIF BACKDROP_PATH=BLANKS,
    POSTER_PATH   CHAR(500)    NULLIF POSTER_PATH=BLANKS,
    HOMEPAGE      CHAR(500)    NULLIF HOMEPAGE=BLANKS
)

```

Fel nem sorolt táblák betöltése SQL Developer GUI-n keresztül történt.

4.3 TITLE_BASICS tisztítása

```

import csv
import datetime

INPUT_PATH = "TMDB_movie_db.csv"
OUTPUT_PATH = "TMDB_movie_db_filter1.csv"
LOG_PATH = "filter_log.txt"

def log(msg):
    print(msg)
    with open(LOG_PATH, "a", encoding="utf-8") as lf:
        lf.write(msg + "\n")

def main():
    total = kept = dropped = 0

    with open(LOG_PATH, "w", encoding="utf-8") as lf:
        lf.write(f"=== Filter started {datetime.datetime.now()}")
    ===\n")
        lf.write(f"Input:  {INPUT_PATH}\nOutput: {OUTPUT_PATH}\n\n")

    with open(INPUT_PATH, "r", encoding="utf-8", errors="ignore",
newline="") as infile, \
        open(OUTPUT_PATH, "w", encoding="utf-8", newline="") as
outfile:

```

```

        reader = csv.DictReader(infile)
        writer = csv.DictWriter(outfile,
fieldnames=reader.fieldnames)
        writer.writeheader()

        for row in reader:
            total += 1

            adult_str = (row.get("adult") or "").strip().lower()
            vote_str = (row.get("vote_count") or "0").strip()

            try:
                vote_count = int(float(vote_str))
            except ValueError:
                vote_count = 0

            if adult_str == "true" or vote_count < 1:
                dropped += 1
                continue

            writer.writerow(row)
            kept += 1

            if total % 100_000 == 0:
                log(f"Processed {total:,} rows... kept {kept:,},
dropped {dropped:,}")

            log("\n=== Filtering complete ===")
            log(f"Total rows read:      {total:,}")
            log(f"Rows kept:                      {kept:,}")
            log(f"Rows dropped:                   {dropped:,}")
            log(f"Output file:                   {OUTPUT_PATH}")
            log(f"Finished at:                   {datetime.datetime.now()} \n")

if __name__ == "__main__":
    main()

```

4.4 USERS generátor

```

internal class Program
{
    static readonly string[] FirstNames = new[]
    {
        "James", "Mary", "John", "Patricia", "Robert", "Jennifer", "Michael", "Linda",
        "William", "Elizabeth", "David", "Barbara", "Richard", "Susan", "Joseph",
        "Jessica", "Thomas", "Sarah", "Charles", "Karen", "Christopher", "Nancy", "D
        aniel", "Lisa",
        "Matthew", "Betty", "Anthony", "Margaret", "Mark", "Sandra", "Donald", "Ashl
        ey", "Steven", "Kimberly", "Paul", "Emily", "Andrew", "Donna", "Joshua", "Mic
        helle", "Kenneth", "Dorothy", "Kevin", "Carol", "Brian", "Amanda", "George",

```

```

"Melissa", "Timothy", "Deborah", "Edward", "Stephanie", "Ronald", "Rebecca"
, "Jason", "Laura", "Jeffrey", "Sharon", "Ryan", "Cynthia", "Jacob", "Kathlee
n", "Gary", "Amy", "Nicholas", "Angela"
};

    static readonly string[] LastNames = new[]{
"Smith", "Johnson", "Williams", "Brown", "Jones", "Garcia", "Miller", "Davis
", "Rodriguez", "Martinez", "Hernandez", "Lopez", "Gonzalez", "Wilson", "And
erson", "Thomas", "Taylor", "Moore", "Jackson", "Martin", "Lee", "Perez", "Th
ompson", "White", "Harris", "Sanchez", "Clark", "Ramirez", "Lewis", "Robinso
n", "Walker", "Young", "Allen", "King", "Wright", "Scott", "Torres", "Nguyen"
, "Hill", "Flores", "Green", "Adams", "Nelson", "Baker", "Hall", "Rivera", "Ca
mpbell", "Mitchell", "Carter", "Roberts"
};

    static readonly string[] Domains = { "gmail.com", "yahoo.com",
"outlook.com", "example.com" };

    static readonly string[] Countries = new[]
{
"US", "US", "US", "GB", "CA", "AU", "NZ", "IE", "HU", "DE", "FR", "ES", "IT", "NL"
, "BE", "DK", "SE", "NO"
};

    static void Main(string[] args)
    {
        string outputPath = args.Length > 0 ? args[0] : "users_10k.csv";
        int count = args.Length > 1 && int.TryParse(args[1], out var
n) ? Math.Max(1, n) : 10000;

        var rnd = new Random(42);
        DateTime startDate = new DateTime(2016, 1, 1);
        DateTime today = DateTime.Today;

        var seenUser = new
HashSet<string>(StringComparer.OrdinalIgnoreCase);

        using var sw = new StreamWriter(outputPath, false, new
UTF8Encoding(encoderShouldEmitUTF8Identifier: true));

        sw.WriteLine("USERNAME;EMAIL;COUNTRY;SIGNUP_DATE;BIRTH_YEAR;NAME");

        for (int i = 1; i <= count; i++)
        {
            string first = Pick(FirstNames, rnd);
            string last = Pick>LastNames, rnd);
            string name = $"{first} {last}";

            string baseSlug = Slug($"{first}.{last}");
            string username = baseSlug + i.ToString("D5",
CultureInfo.InvariantCulture);

            while (!seenUser.Add(username))
                username = baseSlug + rnd.Next(1,
1000000).ToString("D5", CultureInfo.InvariantCulture);

            string email = $"{username}@{Pick(Domains, rnd)}";

```

```

        string country = Pick(Countries, rnd);

        int deltaDays = (today - startDate).Days;
        DateTime signup = startDate.AddDays(rnd.Next(0, deltaDays
+ 1));

        int signupYear = signup.Year;
        int minYear = Math.Max(1900, signupYear - 90);
        int maxYear = signupYear - 13;

        DateTime minDate = new DateTime(1950, 1, 1);
        DateTime maxDate = new DateTime(2007, 1, 1);
        int birthDeltaDays = (maxDate - minDate).Days;

        DateTime birthDate = minDate.AddDays(rnd.Next(0,
birthDeltaDays));

        string line = string.Join(";",
            username,
            email,
            country,
            signup.ToString("yyyy-MM-dd",
CultureInfo.InvariantCulture),
            birthDate.ToString("yyyy-MM-dd",
CultureInfo.InvariantCulture),
            QuoteCsv(name)
        );

        sw.WriteLine(line);
    }

    Console.WriteLine($"Done -> {Path.GetFullPath(outPath)}");
}

static string Pick(string[] arr, Random rnd) =>
arr[rnd.Next(arr.Length)];

static string Slug(string s)
{
    string ascii = s.ToLowerInvariant();
    return Regex.Replace(ascii, @"[^a-z0-9._-]", "");
}

static string QuoteCsv(string value)
{
    string v = value.Replace("\"", "\\\"");
    return $"\"{v}\"";
}
}

```

4.5 WATCHLIST_ITEMS és WATCHLIST generátor

```

internal class Program
{
    const int DefaultNumUsers = 1000;

```

```

const int UserIdMax = 10000;
const int ListsPerUserMin = 1;
const int ListsPerUserMax = 3;
const int ItemsPerListMin = 5;
const int ItemsPerListMax = 50;

static void Main(string[] args)
{
    if (args.Length < 1)
    {
        Console.Error.WriteLine("Usage: dotnet run --
<source_items_csv> [out_watchlists_csv] [out_watchlist_items_csv]
[numUsers]");
        Console.Error.WriteLine("Example:");
        Console.Error.WriteLine(@" dotnet run --
F:\PROGRAMOK\ORACLE\ORADATA\imdb_data\watchlist_items.csv
F:\PROGRAMOK\ORACLE\ORADATA\imdb_data\watchlists.csv
F:\PROGRAMOK\ORACLE\ORADATA\imdb_data\watchlist_items_generated.csv
1000");
        return;
    }

    var srcPath = args[0];
    var outWatchlists = args.Length > 1 ? args[1] :
"watchlists.csv";
    var outItems = args.Length > 2 ? args[2] :
"watchlist_items_generated.csv";
    int numUsers = args.Length > 3 && int.TryParse(args[3], out
var tmp) ? tmp : DefaultNumUsers;

    var rnd = new Random(1234);
    var tconsts = LoadTconsts(srcPath);
    if (tconsts.Count == 0)
    {
        Console.Error.WriteLine("No TCONST values found in the
source CSV.");
        return;
    }

    var userIds = Enumerable.Range(1, UserIdMax).OrderBy(_ =>
rnd.Next()).Take(numUsers).ToList();

    using var wlWriter = new StreamWriter(outWatchlists, false,
new UTF8Encoding(true));
    using var itemsWriter = new StreamWriter(outItems, false, new
UTF8Encoding(true));

    wlWriter.WriteLine("LIST_ID,USER_ID,LIST_NAME,DESCRIPTION,IS_DEFAULT,
IS_PUBLIC,CREATED_AT,UPDATED_AT");

    itemsWriter.WriteLine("LIST_ID,TCONST,ADDED_AT,POSITION_NO,NOTE");

    int nextListId = 1;
    DateTime startCreated = new DateTime(2019, 1, 1);

```

```

DateTime today = DateTime.Today;

foreach (var uid in userIds)
{
    int k = NextInt(rnd, ListsPerUserMin, ListsPerUserMax);
    int defaultIndex = NextInt(rnd, 0, k - 1);

    for (int i = 0; i < k; i++)
    {
        int listId = nextListId++;
        var createdAt = RandDate(rnd, startCreated, today);
        var updatedAt = createdAt.AddDays(NextInt(rnd, 0,
365));

        if (updatedAt > today) updatedAt = today;

        int isPublic = rnd.NextDouble() < 0.7 ? 1 : 0;
        int isDefault = (i == defaultIndex) ? 1 : 0;

        string listName = Pick(rnd, new[]
        {
            "Watch Later", "My Favorites", "Top Picks", "Weekend
Binge",
            "Drama Night", "Action Queue", "Family Time", "Oscar
Nominees",
            "Classics", "Hidden Gems", "Comedy Pack", "Sci-Fi
Marathon"
        }) + $" #{i + 1}";

        string description = Pick(rnd, new[]
        {
            "Autogenerated sample list.",
            "Personal picks.",
            "Randomized seed list.",
            "Movies to check soon.",
            "Mixed genres selection."
        });

        wlWriter.WriteLine(string.Join(", ",
            listId,
            uid,
            QuoteCsv(listName),
            QuoteCsv(description),
            isDefault,
            isPublic,
            createdAt.ToString("yyyy-MM-dd",
CultureInfo.InvariantCulture),
            updatedAt.ToString("yyyy-MM-dd",
CultureInfo.InvariantCulture)
        ));

        int nItems = NextInt(rnd, ItemsPerListMin,
ItemsPerListMax);
        var picks = SampleWithoutReplacement(rnd, tconsts,
Math.Min(nItems, tconsts.Count));

        int pos = 1;

```

```

        foreach (var t in picks)
        {
            var addedAt = createdAt.AddDays(NextInt(rnd, 0,
60));

            if (addedAt > today) addedAt = today;

            string note = Pick(rnd, new[]
            {
                "", "Looks interesting", "Recommended by a
friend", "From top rated list", "Rewatch"
            });

            itemsWriter.WriteLine(string.Join(",",
                listId,
                t,
                addedAt.ToString("yyyy-MM-dd",
CultureInfo.InvariantCulture),
                pos,
                QuoteCsv(note)
            ));
            pos++;
        }
    }

    Console.WriteLine("Done");
    Console.WriteLine("WATCHLISTS -> " +
Path.GetFullPath(outWatchlists));
    Console.WriteLine("ITEMS -> " +
Path.GetFullPath(outItems));
}

static List<string> LoadTconsts(string path)
{
    var result = new List<string>();
    using var sr = new StreamReader(path, Encoding.UTF8, true);
    string? header = sr.ReadLine();
    if (header == null) return result;

    var cols = header.Split(',');
    int idx = Array.FindIndex(cols, c =>
c.Trim().Equals("tconst", StringComparison.OrdinalIgnoreCase));
    if (idx < 0) idx = 0;

    string? line;
    while ((line = sr.ReadLine()) != null)
    {
        if (string.IsNullOrWhiteSpace(line)) continue;
        var parts = SplitCsv(line);
        if (idx < parts.Count)
        {
            var v = parts[idx].Trim();
            if (!string.IsNullOrEmpty(v)) result.Add(Unquote(v));
        }
    }
}

```

```

        return result.Distinct(StringComparer.Ordinal).ToList();
    }

    static List<string> SplitCsv(string line)
    {
        var res = new List<string>();
        var sb = new StringBuilder();
        bool inQuotes = false;
        for (int i = 0; i < line.Length; i++)
        {
            char ch = line[i];
            if (inQuotes)
            {
                if (ch == '"')
                {
                    if (i + 1 < line.Length && line[i + 1] == '"')
                    {
                        sb.Append('"'); i++;
                    }
                    else
                    {
                        inQuotes = false;
                    }
                }
                else sb.Append(ch);
            }
            else
            {
                if (ch == ',')
                {
                    res.Add(sb.ToString()); sb.Clear();
                }
                else if (ch == '"')
                {
                    inQuotes = true;
                }
                else sb.Append(ch);
            }
        }
        res.Add(sb.ToString());
        return res;
    }

    static string Unquote(string s) => (s.StartsWith("\"") &&
s.EndsWith("\"")) ? s.Substring(1, s.Length - 2).Replace("\"\"",
"") : s;

    static string QuoteCsv(string value)
    {
        if (string.IsNullOrEmpty(value)) return "";
        string v = value.Replace("\"", "\"\"");
        return $"\"{v}\"";
    }

```

```

    static T Pick<T>(Random rnd, IReadOnlyList<T> arr) =>
arr[rnd.Next(arr.Count)];

    static int NextInt(Random rnd, int minInclusive, int
maxInclusive) => minInclusive + rnd.Next(maxInclusive - minInclusive
+ 1);

    static DateTime RandDate(Random rnd, DateTime start, DateTime
end)
    {
        int days = (end - start).Days;
        return start.AddDays(rnd.Next(days + 1));
    }

    static List<T> SampleWithoutReplacement<T>(Random rnd,
IReadOnlyList<T> population, int k)
    {
        var idx = Enumerable.Range(0, population.Count).ToList();
        for (int i = 0; i < k; i++)
        {
            int j = i + rnd.Next(idx.Count - i);
            (idx[i], idx[j]) = (idx[j], idx[i]);
        }
        var res = new List<T>(k);
        for (int i = 0; i < k; i++) res.Add(population[idx[i]]);
        return res;
    }
}

```